

Unit-8 Unix/Linux Operating System

Development of Unix/Linux

According to the GTU Operating Systems syllabus, "Development of Unix/Linux" is an important topic in Unit-8: Unix/Linux Operating System.

1. Introduction

Unix and Linux are powerful multiuser and multitasking operating systems widely used in servers, cloud computing, networking, cybersecurity, embedded systems, and software development.

- **Unix** was developed first as a research operating system.
- **Linux** was later developed inspired by Unix concepts.
- Today, Linux powers:
 - Web servers
 - Android phones
 - Supercomputers
 - Cloud platforms
 - DevOps systems

2. Definition

Term	Definition
Unix	A multiuser, multitasking operating system developed at Bell Labs in 1969
Linux	An open-source Unix-like operating system kernel developed by Linus Torvalds in 1991

3. History of Unix

3.1 Birth of Unix (1969)

- Developed at **AT&T Bell Laboratories**
- Main developers:
 - Ken Thompson
 - Dennis Ritchie

Reason for Development

Researchers wanted:

- Simpler operating system
- Better multitasking support
- Multiuser environment

3.2 Important Milestones of Unix

Year	Development
1969	First version of Unix created
1973	Unix rewritten in C language
1975	Unix distributed to universities
1980s	Commercial Unix versions released
1990s	Unix used widely in servers and workstations

4. Evolution of Linux

4.1 Beginning of Linux (1991)

Linux was developed by:

- Linus Torvalds

Objective

To create:

- Free operating system
- Unix-like environment
- Open-source kernel for personal computers

4.2 GNU Project Contribution

The GNU Project provided:

- Compiler
- Libraries
- Utilities
- Shell tools

Combination:

- GNU tools + Linux kernel = GNU/Linux Operating System

4.3 Growth of Linux

Period	Development
1991	Linux Kernel 0.01 released
1994	Stable Linux version released
2000s	Linux adopted in servers
2008 onward	Android based on Linux kernel
Present	Linux dominates cloud and supercomputing

5. Chronological Development Timeline

1969 → Unix developed at Bell Labs

1973 → Unix rewritten in C

1980s → Unix commercial systems

1991 → Linux kernel created

1994 → Stable Linux release

2000s → Linux server growth

2008 → Android uses Linux kernel

Today → Linux powers cloud, AI, IoT, supercomputers

6. Working Principle of Unix/Linux

Unix/Linux works using interaction between:

User



Shell



Kernel

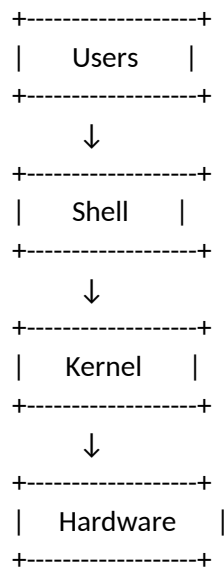


Hardware

Explanation

Component	Role
User	Gives commands
Shell	Interprets commands
Kernel	Controls hardware/resources
Hardware	Executes operations

7. Basic Architecture of Unix/Linux



8. Features of Unix/Linux

Feature	Explanation
Multituser	Multiple users can work simultaneously
Multitasking	Multiple processes run together
Portability	Works on different hardware
Security	User permissions and protection
Stability	Reliable for long-running systems
Hierarchical File System	Tree-like directory structure
Shell Scripting	Automation support
Open Source (Linux)	Source code freely available
Networking	Strong network capabilities

9. Unix vs Linux Comparison

Feature	Unix	Linux
Developed By	AT&T Bell Labs	Linus Torvalds
Year	1969	1991
Source Code	Mostly proprietary	Open source
Cost	Expensive	Mostly free
Usage	Enterprise systems	Servers, cloud, desktops
Flexibility	Limited	Highly customizable

10. Advantages & Disadvantages

Advantages

Advantages	Explanation
High Stability	Rarely crashes
Secure	Strong user control
Efficient Multitasking	Handles many tasks
Powerful Shell	Easy automation
Open-source Support	Linux community development

Disadvantages

Disadvantages	Explanation
---------------	-------------

Command Complexity	Difficult for beginners
Hardware Driver Issues	Some devices unsupported
Learning Curve	Requires command-line knowledge
Commercial Unix Cost	Expensive licenses

11. Real-World Examples

Area	Usage
Web Servers	Linux servers host websites
Android OS	Uses Linux kernel
Supercomputers	Most run Linux
Cloud Computing	AWS, Azure, Google Cloud use Linux
Networking Devices	Routers and switches use Unix/Linux

12. Key Terms

Term	Meaning
Kernel	Core part of operating system
Shell	Command interpreter
Open Source	Source code publicly available
Multitasking	Running multiple programs
Multuser	Multiple users access system
Distribution (Distro)	Linux operating system package
GNU	Free software project

13. Important Exam Points

Very Important Theory Questions

1. Explain development of Unix and Linux.
2. Differentiate Unix and Linux.
3. Explain features of Unix/Linux.
4. Write chronological evolution of Linux.
5. Explain Unix/Linux architecture.

One-Line Exam Definitions

Topic	Definition
Unix	Multuser multitasking operating system
Linux	Open-source Unix-like operating system
Kernel	Core manager of system resources
Shell	Interface between user and kernel

14. Short Conclusion

Unix introduced the foundation of modern operating systems with concepts like multitasking and multuser support. Linux extended these ideas with open-source development and flexibility. Today, Unix/Linux systems are essential in servers, cloud computing, networking, cybersecurity, and modern software infrastructure.

Unix/Linux Architecture

According to the GTU Operating Systems syllabus, "Unix/Linux Architecture" is an important Unit-8 topic that includes Kernel, Shell, File System, and Utilities.

1. Introduction

Unix/Linux architecture defines how different parts of the operating system interact with users and hardware.

The architecture is layered:

- User interacts with the system
- Shell interprets commands
- Kernel manages hardware/resources
- Hardware performs actual operations

Unix/Linux architecture is known for:

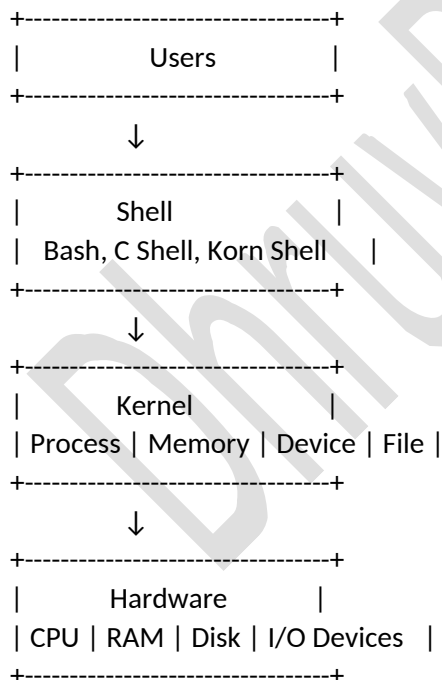
- Simplicity
- Stability
- Security
- Modular design

2. Definition

Component	Definition
Unix/Linux Architecture	Structured organization of OS components such as Kernel, Shell, File System, and Utilities
Kernel	Core component that controls hardware and system resources
Shell	Command interpreter between user and kernel
File System	Method of organizing and storing files
Utilities	System programs/tools used for operations

3. Overall Unix/Linux Architecture

Architecture Diagram



4. Working Principle of Unix/Linux Architecture

Step-by-Step Flow

Step Operation

- 1 User enters command

Step Operation

- 2 Shell interprets command
- 3 Shell sends request to kernel
- 4 Kernel communicates with hardware
- 5 Hardware performs task
- 6 Result returned to user

Example

Command:

ls

Working:

1. User types ls
2. Shell interprets command
3. Kernel accesses file system
4. Directory contents fetched
5. Output displayed

5. Kernel in Unix/Linux

5.1 Introduction to Kernel

Kernel is the heart of Unix/Linux OS.

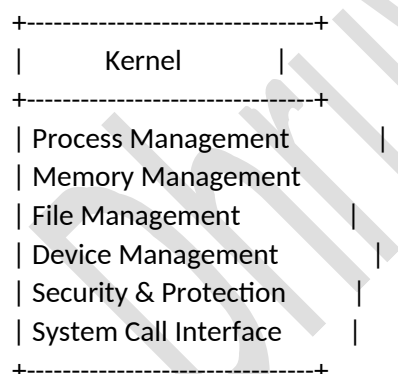
It:

- Manages hardware
- Controls system resources
- Provides interface between software and hardware

Without kernel:

- Operating system cannot function

5.2 Kernel Architecture Diagram



5.3 Functions of Kernel

A. Process Management

Kernel:

- Creates processes
- Schedules CPU
- Terminates processes
- Handles multitasking

Example

Running:
chrome + music + terminal
simultaneously.

B. Memory Management

Kernel:

- Allocates RAM
- Frees unused memory
- Handles virtual memory
- Prevents memory conflicts

C. File System Management

Kernel:

- Creates/deletes files
- Manages directories
- Handles permissions
- Controls disk access

D. Device Management

Kernel controls:

- Keyboard
- Mouse
- Printer
- Disk
- USB devices

Using:

- Device drivers

E. Security Management

Kernel provides:

- User authentication
- Permission control
- Access protection

F. System Call Handling

Kernel executes:

- fork()
- exec()
- read()
- write()

System calls allow user programs to access OS services.

5.4 Types of Kernels

Type	Description
Monolithic Kernel	All services inside kernel space
Microkernel	Minimal services inside kernel
Hybrid Kernel	Combination of both

Linux Kernel Type

Linux mainly uses:

- Monolithic kernel architecture

5.5 Advantages of Kernel

Advantages
Efficient resource management
Supports multitasking
Hardware abstraction
Security enforcement
Stable system operation

5.6 Disadvantages of Kernel

Disadvantages
Kernel crash affects whole system
Complex development
Large kernel size in monolithic systems

6. Shell in Unix/Linux

6.1 Introduction to Shell

Shell is a command interpreter.

It acts as:

- Interface between user and kernel

User never directly communicates with kernel.

6.2 Shell Working Diagram

User Command



Shell



Kernel



Hardware

6.3 Functions of Shell

Function	Description
Command Interpretation	Understands commands
Program Execution	Runs applications
Shell Scripting	Automates tasks
File Manipulation	Handles files/directories
Environment Control	Manages variables

6.4 Types of Shell

Shell	Description
Bourne Shell (sh)	Original Unix shell
Bash	Most common Linux shell
C Shell (csh)	C-like syntax
Korn Shell (ksh)	Enhanced shell
Z Shell (zsh)	Advanced modern shell

6.5 Example Commands

Command	Purpose
pwd	Show current directory
ls	List files
mkdir	Create directory
rm	Remove file
cd	Change directory

6.6 Shell Script Example

echo "Hello Linux"

Output

Hello Linux

6.7 Advantages of Shell

Advantages
Easy command execution
Supports automation
Powerful scripting
Fast system control

6.8 Disadvantages of Shell

Disadvantages
Command syntax difficult
Typing mistakes cause errors
Less user-friendly for beginners

7. File System in Unix/Linux

7.1 Introduction

Unix/Linux stores everything as files.

Examples:

- Text files
- Programs
- Devices
- Directories

7.2 Hierarchical File System Structure

/ (Root)

├─ bin

├─ etc

├─ home

├─ usr

├─ var

├─ dev

└─ tmp

7.3 Important Directories

Directory	Purpose
-----------	---------

/	Root directory
/bin	Essential commands
/etc	Configuration files
/home	User files
/usr	User applications
/var	Log and variable files
/dev	Device files
/tmp	Temporary files

7.4 Features of Unix/Linux File System

Feature	Explanation
Hierarchical Structure	Tree-like organization
File Permissions	Security control
Device as File	Hardware treated as files
Case Sensitive	File names are case-sensitive

7.5 File Permissions

Permission Format

rwX rwX rwX

Symbol	Meaning
r	Read
w	Write
x	Execute

7.6 Advantages of File System

Advantages
Organized data storage
Better security
Efficient file access
Easy backup management

7.7 Disadvantages of File System

Disadvantages
Permission management complexity
Requires command knowledge
Large systems become difficult to manage

8. Utilities in Unix/Linux

8.1 Introduction

Utilities are system programs/tools used to perform specific operations.

Utilities help users:

- Manage files
- Monitor system
- Edit text
- Process data

8.2 Common Unix/Linux Utilities

Utility	Purpose
---------	---------

ls	List files
cp	Copy files
mv	Move files
grep	Search text
cat	Display file content
ps	Show processes
top	System monitoring
chmod	Change permissions

8.3 Utility Working

Example:

cp file1.txt file2.txt

Operation

- Utility requests kernel
- Kernel performs disk operation
- File copied successfully

8.4 Categories of Utilities

Category	Examples
File Utilities	cp, mv, rm
Text Utilities	grep, cat
Process Utilities	ps, kill
Disk Utilities	df, du
Network Utilities	ping, ssh

8.5 Advantages of Utilities

Advantages
Simplifies tasks
Increases productivity
Supports automation
Powerful command-line control

8.6 Disadvantages of Utilities

Disadvantages
Requires command knowledge
Wrong commands may delete data
Complex syntax for beginners

9. Complete Interaction Between Components

Full Workflow Example

Command:

cat file.txt

Internal Working

Component	Operation
User	Enters command
Shell	Interprets command
Utility (cat)	Requests file access
Kernel	Reads file from disk

File System	Locates file blocks
Hardware	Retrieves data
Output	Displayed on screen

10. Comparison Table

Kernel vs Shell vs Utilities

Feature	Kernel	Shell	Utilities
Type	Core OS component	Command interpreter	System tools
Role	Controls hardware	User interface	Perform tasks
Runs In	Kernel space	User space	User space
Importance	Essential	Important	Supportive
Example	Process manager	Bash	ls, cp, grep

11. Real-World Examples

System	Usage
Ubuntu Linux	Desktop operating system
Android	Linux kernel based
Web Servers	Apache/Nginx on Linux
Cloud Systems	AWS Linux servers
Supercomputers	Mostly Linux based

12. Key Terms

Term	Meaning
Kernel Space	Protected memory area
User Space	Area for user programs
Shell Script	Automated command file
Device Driver	Software controlling hardware
Root Directory	Top directory /
System Call	Interface to kernel

13. Important Exam Points

Most Important Questions

1. Explain Unix/Linux architecture with diagram.
2. Explain role and functions of kernel.
3. Differentiate kernel and shell.
4. Explain Unix/Linux file system structure.
5. Explain shell and types of shell.
6. Explain utilities in Unix/Linux.
7. Explain interaction between shell and kernel.

14. Short Conclusion

Unix/Linux architecture follows a layered structure where the shell acts as an interface between users and the kernel. The kernel manages system resources, the file system organizes data, and utilities provide useful operations. This modular design makes Unix/Linux stable, secure, flexible, and widely used in modern computing environments.

Role & Functions of Kernel

According to the GTU Operating Systems syllabus, "Role & Functions of Kernel" is one of the highest-priority topics in Unit-8: Unix/Linux Operating System.

1. Introduction

The kernel is the most important part of an operating system.

It acts as a bridge between:

- Hardware
- Software
- Users

Every operation in Unix/Linux such as:

- Running programs
- Managing memory
- Accessing files
- Using devices

is controlled by the kernel.

Without the kernel:

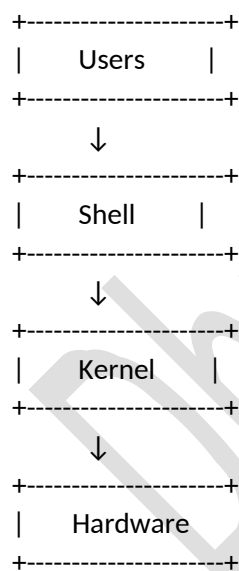
- The operating system cannot work.

2. Definition of Kernel

Term	Definition
Kernel	The core component of an operating system that manages hardware resources and provides services to programs

3. Position of Kernel in Operating System

Basic Architecture Diagram



4. Working Principle of Kernel

Step-by-Step Working

Step	Operation
1	User gives command
2	Shell interprets command
3	Request sent to kernel
4	Kernel communicates with hardware

5	Hardware performs operation
6	Result returned to user

Example

Command:

cat file.txt

Internal Operation

1. User enters command
2. Shell interprets it
3. Kernel accesses file system
4. Disk retrieves data
5. Output displayed

5. Main Functions of Kernel

The kernel performs four major functions:

1. Process Management
2. Memory Management
3. File System Management
4. Device Management

6. Process Management

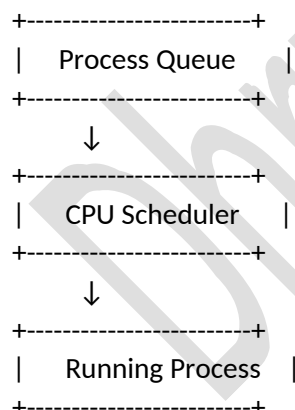
6.1 Introduction

A process is a program in execution.

Kernel manages:

- Creation of processes
- Scheduling of CPU
- Execution control
- Process termination

6.2 Process Management Diagram



6.3 Functions of Kernel in Process Management

Function	Description
Process Creation	Starts new process
CPU Scheduling	Allocates CPU time
Context Switching	Switches between processes
Process Synchronization	Coordinates processes

Process Termination	Ends completed process
----------------------------	------------------------

6.4 Process States

New → Ready → Running → Waiting → Terminated

State	Meaning
New	Process created
Ready	Waiting for CPU
Running	Executing
Waiting	Waiting for resource
Terminated	Process finished

6.5 Real-World Example

Running:

- Browser
- Music player
- Terminal

simultaneously is possible because kernel manages multiple processes.

6.6 Advantages of Process Management

Advantages
Supports multitasking
Better CPU utilization
Faster execution
Multiple users supported

6.7 Disadvantages

Disadvantages
Context switching overhead
Complex scheduling
Synchronization issues

7. Memory Management

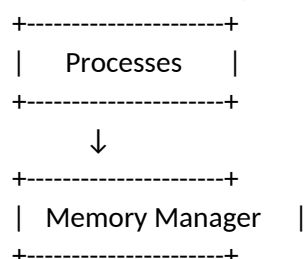
7.1 Introduction

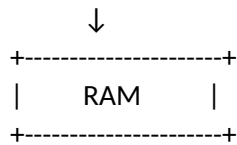
Memory management controls:

- RAM allocation
- Deallocation
- Virtual memory
- Protection

Kernel ensures efficient memory usage.

7.2 Memory Management Diagram





7.3 Functions of Kernel in Memory Management

Function	Description
Memory Allocation	Assigns memory
Memory Deallocation	Frees memory
Virtual Memory	Uses disk as extra memory
Protection	Prevents illegal access
Paging	Divides memory into pages

7.4 Virtual Memory Concept

When RAM becomes full:

- Kernel uses hard disk space as temporary memory.

This is called:

- Virtual Memory

7.5 Paging Concept

Memory divided into:

- Pages
- Frames

Kernel maps pages to frames for efficient execution.

7.6 Real-World Example

Opening many browser tabs:

- Kernel dynamically allocates memory.

7.7 Advantages

Advantages
Efficient memory usage
Supports large programs
Prevents memory conflicts
Improves multitasking

7.8 Disadvantages

Disadvantages
Paging overhead
Thrashing problem
Virtual memory slower than RAM

8. File System Management

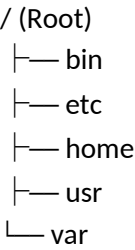
8.1 Introduction

Kernel manages:

- Files
- Directories
- Storage devices
- Access permissions

Unix/Linux treats everything as a file.

8.2 File System Structure



8.3 Functions of Kernel in File Management

Function	Description
File Creation	Creates files
File Deletion	Removes files
Read/Write Operations	Accesses data
Directory Management	Organizes files
Permission Control	Provides security

8.4 File Permissions

rwX rwX rwX

Symbol	Meaning
r	Read
w	Write
x	Execute

8.5 File System Types in Linux

File System	Usage
ext4	Common Linux file system
FAT	Compatible with Windows
NTFS	Windows file system
XFS	High-performance system

8.6 Real-World Example

Saving documents in:
/home/user/Documents

Kernel handles:

- Storage
- Permissions
- Retrieval

8.7 Advantages

Advantages
Organized storage
Better security
Efficient access
Reliable management

8.8 Disadvantages

Disadvantages
Corruption risk
Permission complexity
Disk fragmentation

9. Device Management

9.1 Introduction

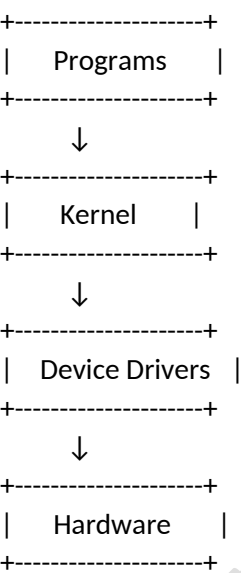
Kernel controls all hardware devices using:

- Device drivers

Examples:

- Keyboard
- Mouse
- Printer
- USB
- Hard disk

9.2 Device Management Diagram



9.3 Functions of Kernel in Device Management

Function	Description
Device Communication	Talks to hardware
Driver Management	Uses device drivers
Input/Output Control	Handles I/O operations
Buffering	Temporary data storage
Interrupt Handling	Responds to hardware signals

9.4 Device Drivers

A device driver is software that helps the kernel communicate with hardware.

Examples:

- Printer driver
- Audio driver
- Graphics driver

9.5 Interrupt Handling

When hardware needs CPU attention:

- It sends interrupt signal
- Kernel handles request immediately

9.6 Real-World Example

Connecting USB drive:

1. Kernel detects hardware
2. Driver loaded
3. Device mounted automatically

9.7 Advantages

Advantages
Hardware abstraction
Efficient device usage
Automatic device handling
Better I/O management

9.8 Disadvantages

Disadvantages
Driver compatibility issues
Device conflicts
Hardware dependency

10. Complete Functional Overview of Kernel Integrated Kernel Operation

Kernel
Process Management
Memory Management
File System Management
Device Management
Security & Protection
System Call Interface

11. Types of Kernel

Kernel Type	Description
Monolithic Kernel	All services inside kernel
Microkernel	Minimal services in kernel
Hybrid Kernel	Combination approach

Linux Kernel Type

Linux mainly uses:

- Monolithic kernel architecture

12. Comparison Table

Process vs Memory vs File vs Device Management

Function	Purpose	Example
Process Management	Manage execution	Running apps
Memory Management	Manage RAM	Virtual memory
File Management	Organize files	Saving documents
Device Management	Control hardware	USB detection

13. Advantages of Kernel

Advantages
Resource management
Hardware control
Multitasking support
Security enforcement
Efficient execution

14. Disadvantages of Kernel

Disadvantages
Complex design
Kernel crash affects entire system
Difficult debugging
Large memory usage

15. Key Terms

Term	Meaning
Process	Program in execution
Scheduler	Allocates CPU
Paging	Memory division technique
Virtual Memory	Disk used as RAM
Device Driver	Hardware communication software
Interrupt	Hardware attention signal
Context Switching	Switching CPU between processes

17. Short Conclusion

The kernel is the core of Unix/Linux operating systems and is responsible for managing processes, memory, files, and hardware devices. It acts as the communication bridge between software and hardware, ensuring efficient resource utilization, multitasking, security, and stable system operation.

System Calls in Unix/Linux

According to the GTU Operating Systems syllabus, "System Calls" is a very high-priority topic in Unit-8: Unix/Linux Operating System. Important examples include `fork()`, `exec()`, `open()`, `read()`, and `write()`.

1. Introduction

A user program cannot directly access hardware resources such as:

- CPU
- Memory
- Files
- Devices

To access operating system services, programs use:

- System Calls

System calls act as an interface between:

- User programs
- Operating system kernel

2. Definition

Term	Definition
System Call	A request made by a user program to the operating system kernel for performing specific operations

3. Need of System Calls

Programs require system calls for:

- File handling
- Process creation
- Memory access
- Device communication
- Input/output operations

Without system calls:

- Applications cannot interact with hardware safely.

4. Working Principle of System Calls

Basic Flow

User Program



System Call Interface



Kernel



Hardware

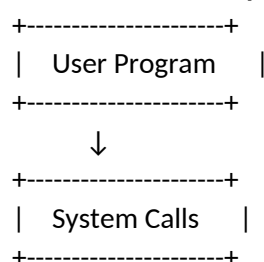
Step-by-Step Working

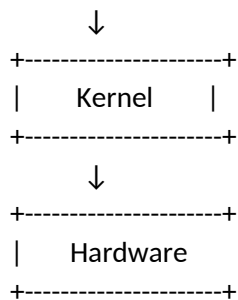
Step	Operation
1	User program requests service
2	System call invoked
3	CPU switches to kernel mode
4	Kernel performs requested task
5	Result returned to user program

Example

Reading a File : Program → read() → Kernel → Disk → Data Returned

5. Architecture of System Calls





6. Types of System Calls

Type	Purpose
Process Control	Create/manage processes
File Management	Handle files
Device Management	Manage hardware devices
Information Maintenance	System information
Communication	IPC and networking

7. Process Control System Calls

These system calls manage processes.

Examples:

- `fork()`
- `exec()`
- `wait()`
- `exit()`

8. `fork()` System Call

8.1 Definition

`fork()` creates a new process called:

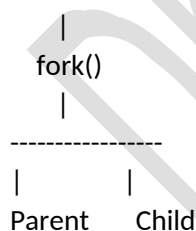
- Child Process

The original process is:

- Parent Process

8.2 Working of `fork()`

Parent Process



8.3 Syntax

```
pid_t pid = fork();
```

8.4 Return Values

Return Value	Meaning
0	Child process
Positive	Parent process receives child PID
Negative	Fork failed

8.5 Example Program

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();
    printf("Process Created\n");
    return 0;
}
```

8.6 Output

Process Created

Process Created

Because:

- Parent and child both execute the statement.

8.7 Real-World Example

Linux creates child processes when:

- Opening applications
- Running commands in terminal

8.8 Advantages

Advantages
Supports multitasking
Enables parallel execution
Process isolation

8.9 Disadvantages

Disadvantages
High memory overhead
Context switching cost

9. exec() System Call

9.1 Definition

exec() replaces the current process image with a new program.

It is generally used after fork().

9.2 Working of exec()

Process



exec()



New Program Loaded

9.3 Common exec Variants

Variant	Description
execl()	Argument list

execv()	Argument array
execlp()	Uses PATH variable
execvp()	Array + PATH

9.4 Example Program

```
#include<stdio.h>
```

```
#include<unistd.h>
```

```
int main()
{
    execl("/bin/ls","ls",NULL);
    return 0;
}
```

9.5 Operation

- Current process replaced by:
 - ls command

9.6 Real-World Example

Terminal internally uses:

- fork() + exec()

to run commands.

10. open() System Call

10.1 Definition

open() opens or creates a file.

Used before:

- Reading
- Writing

10.2 Syntax

```
int fd = open("file.txt", O_RDONLY);
```

10.3 Parameters

Parameter	Meaning
File Name	File to open
Mode	Read/write mode

10.4 Modes

Mode	Purpose
O_RDONLY	Read only
O_WRONLY	Write only
O_RDWR	Read and write

10.5 Example

```
int fd = open("data.txt", O_RDONLY);
```

10.6 Real-World Example

Opening:

- Text files
- Images
- Documents

11. read() System Call

11.1 Definition

read() reads data from:

- File
- Keyboard
- Device

11.2 Syntax

read(fd, buffer, size);

11.3 Parameters

Parameter	Meaning
fd	File descriptor
buffer	Data storage
size	Number of bytes

11.4 Example Program

```
char buf[100];
read(fd, buf, 100);
```

11.5 Working

Disk/File



Kernel



Buffer

11.6 Real-World Example

Reading:

- Files
- Keyboard input
- USB data

12. write() System Call

12.1 Definition

write() writes data to:

- File
- Screen
- Device

12.2 Syntax

write(fd, buffer, size);

12.3 Example Program

```
write(fd, "Hello", 5);
```

12.4 Working

Buffer



Kernel



Disk/File

12.5 Real-World Example

Saving:

- Documents
- Program output
- Logs

13. Complete System Call Execution Example

Command Execution Flow

User Program



fork()



Child Process



exec()



Program Loaded



open()



read()/write()



Output Generated

14. Comparison Table of Important System Calls

System Call	Purpose	Category
fork()	Create process	Process control
exec()	Replace process	Process control
open()	Open file	File management
read()	Read data	File management
write()	Write data	File management

15. Advantages of System Calls

Advantages
Safe hardware access
Controlled resource management
Process communication
File management support
Security protection

16. Disadvantages of System Calls

Disadvantages
Kernel mode switching overhead
Slower than direct access
Complex programming

17. Real-World Applications

Area	Usage
Linux Terminal	Command execution
Web Servers	Process creation
File Editors	Reading/writing files
Operating Systems	Resource management
Databases	File handling

18. Key Terms

Term	Meaning
System Call	Request to kernel
Kernel Mode	Privileged execution mode
User Mode	Restricted execution mode
File Descriptor	Integer representing open file
Child Process	New process created by fork()
Parent Process	Original process

20. Short Conclusion

System calls provide the communication interface between user programs and the operating system kernel. They enable safe access to hardware and operating system services such as process creation, file handling, memory access, and device communication. Important system calls like fork(), exec(), open(), read(), and write() form the foundation of Unix/Linux operating systems.

Elementary Linux Commands

According to the GTU Operating Systems syllabus, "Elementary Linux Commands" is an important topic in Unit-8: Unix/Linux Operating System.

1. Introduction

Linux is a command-line based operating system where users interact using commands.

Linux commands are used for:

- File management
- Directory handling
- Navigation
- Copying/moving files
- Deleting files

These commands are executed through:

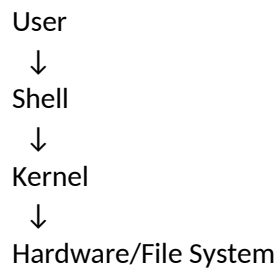
- Shell (Terminal)

2. Definition

Term	Definition
Linux Command	Instruction given to Linux shell to perform specific operation
Shell	Command interpreter between user and kernel
Terminal	Interface used to execute commands

3. Working Principle of Linux Commands

Basic Working Flow



Example

Command:

ls

Operation

1. User enters command
2. Shell interprets command
3. Kernel accesses file system
4. Directory contents displayed

4. Command Structure in Linux

General Syntax

command [options] [arguments]

Example

ls -l Documents

Part	Meaning
ls	Command
-l	Option
Documents	Argument

5. pwd Command

5.1 Definition

pwd means:

- Print Working Directory

It displays:

- Current directory path

5.2 Syntax

pwd

5.3 Example

\$ pwd

/home/student

5.4 Working Principle

Current Directory Path



Displayed on Terminal

6. ls Command

6.1 Definition

ls means:

- List directory contents

Used to display:

- Files
- Directories

6.2 Syntax

ls

6.3 Common Options

Option	Purpose
ls -l	Long listing
ls -a	Hidden files
ls -lh	Human-readable size

6.4 Example

ls -l

6.5 Sample Output

-rw-r--r-- file1.txt

drwxr-xr-x Documents

7. cd Command

7.1 Definition

cd means:

- Change Directory

Used to move between directories.

7.2 Syntax

cd directory_name

7.3 Examples

Move to Documents

cd Documents

Move to Parent Directory

cd ..

Move to Home Directory

cd ~

7.4 Working Principle

Current Directory



Change Requested



New Directory Activated

8. mkdir Command

8.1 Definition

mkdir means:

- Make Directory

Used to create new directories.

8.2 Syntax

mkdir directory_name

8.3 Example

mkdir Project

8.4 Creating Multiple Directories

mkdir dir1 dir2 dir3

9. rmdir Command

9.1 Definition

rmdir means:

- Remove Directory

Used to delete:

- Empty directories

9.2 Syntax

rmdir directory_name

9.3 Example

rmdir Project

9.4 Important Note

rmdir works only if directory is empty.

10. cp Command

10.1 Definition

cp means:

- Copy files/directories

10.2 Syntax

cp source destination

10.3 Example

cp file1.txt backup.txt

10.4 Copy Directory

cp -r folder1 folder2

10.5 Working Principle

Source File



Duplicate Created



Destination Location

11. mv Command

11.1 Definition

mv means:

- Move or rename files/directories

11.2 Syntax

mv source destination

11.3 Example: Move File

mv file.txt Documents/

11.4 Example: Rename File

mv old.txt new.txt

12. rm Command

12.1 Definition

rm means:

- Remove files/directories

12.2 Syntax

rm filename

12.3 Example

rm data.txt

12.4 Remove Directory Recursively

rm -r folder

12.5 Force Delete

rm -f file.txt

12.6 Working Principle

Selected File



Kernel Removes Entry



Storage Released

13. Complete Command Workflow Example

Example Operations

mkdir Demo

cd Demo

touch file.txt

cp file.txt copy.txt

mv copy.txt new.txt

ls

rm new.txt

pwd

Internal Working

User Command



Shell



Kernel



File System Updated

14. Comparison Table of Linux Commands

Command	Purpose	Example
pwd	Show current directory	pwd
ls	List files	ls -l
cd	Change directory	cd Documents
mkdir	Create directory	mkdir Test
rmdir	Remove empty directory	rmdir Test
cp	Copy files	cp a.txt b.txt
mv	Move/rename files	mv old.txt new.txt
rm	Delete files	rm file.txt

15. Advantages of Linux Commands

Advantages
Fast execution
Powerful control
Automation support
Efficient file management
Low resource usage

16. Disadvantages of Linux Commands

Disadvantages
Difficult for beginners
Wrong commands may delete data
Requires syntax knowledge

17. Real-World Examples

Area	Usage
System Administration	File management
DevOps	Server handling
Programming	Managing project files
Cybersecurity	Terminal operations
Cloud Computing	Linux server management

18. Key Terms

Term	Meaning
Directory	Folder in Linux
Root Directory	Top directory /

Shell	Command interpreter
Terminal	Command execution interface
Recursive	Repeating operation on subfolders
Path	File/directory location

19. Important Exam Points

Most Important Questions

1. Explain basic Linux commands.
2. Explain ls command with options.
3. Explain cd and pwd.
4. Explain cp, mv, and rm.
5. Differentiate mkdir and rmdir.
6. Write syntax and examples of Linux commands.

High-Probability Commands

Priority	Command
Very High	ls
Very High	cd
High	cp
High	mv
High	rm
Medium	mkdir
Medium	rmdir
Medium	pwd

Frequently Asked Short Notes

- pwd
- ls
- cd
- mkdir
- rm
- cp
- mv

20. Short Conclusion

Elementary Linux commands provide the foundation for interacting with Unix/Linux operating systems. Commands such as ls, cd, pwd, mkdir, cp, mv, and rm help users navigate directories, manage files, and perform system operations efficiently through the command-line interface.

Shell Programming

According to the GTU Operating Systems syllabus, "Shell Programming" is an important topic in Unit-8: Unix/Linux Operating System. It includes Shell Concept, Basic Shell Scripting, Variables, and Simple Scripts.

1. Introduction

Shell programming is used to automate tasks in Unix/Linux systems.

Instead of typing commands repeatedly:

- We write commands inside a file

- The shell executes them automatically

Shell programming helps in:

- Automation
- System administration
- File management
- Process execution

2. Definition

Term	Definition
Shell	Command interpreter between user and kernel
Shell Script	File containing Linux commands executed sequentially
Shell Programming	Writing scripts to automate Linux tasks

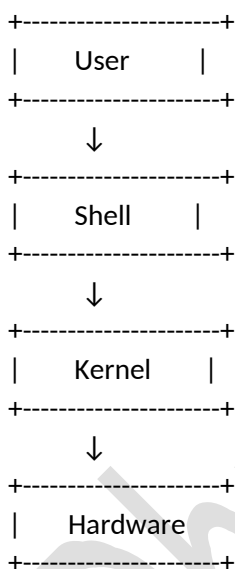
3. Shell Concept

The shell acts as an interface between:

- User
- Operating System Kernel

Users enter commands in shell, and shell communicates with kernel.

4. Shell Architecture



5. Working Principle of Shell

Step-by-Step Process

Step	Operation
1	User enters command
2	Shell interprets command
3	Shell sends request to kernel
4	Kernel executes operation
5	Output returned to shell
6	Result displayed to user

Example

Command : ls

Working: User → Shell → Kernel → File System → Output

6. Types of Shell

Shell	Description
Bourne Shell (sh)	Original Unix shell
Bash	Bourne Again Shell (most common)
C Shell (csh)	C-like syntax
Korn Shell (ksh)	Improved shell
Z Shell (zsh)	Advanced modern shell

Most Common Shell in Linux

- Bash (Bourne Again Shell)

7. Shell Scripting

7.1 Introduction

A shell script is:

- A text file containing Linux commands

Scripts are executed by shell automatically.

7.2 Advantages of Shell Scripts

Advantages
Task automation
Saves time
Reduces manual work
Easy system administration
Supports batch processing

7.3 Disadvantages

Disadvantages
Slower than compiled languages
Syntax errors common
Difficult debugging

8. Structure of Shell Script

Basic Structure

```
#!/bin/bash
```

```
# Commands
```

```
echo "Hello Linux"
```

Explanation

Part	Meaning
#!/bin/bash	Bash shell interpreter
#	Comment
echo	Display output

9. Steps to Execute Shell Script

Step 1: Create Script

```
nano script.sh
```

Step 2: Write Script

```
#!/bin/bash  
echo "Welcome to Linux"
```

Step 3: Give Permission

```
chmod +x script.sh
```

Step 4: Execute Script

```
./script.sh
```

Output

Welcome to Linux

10. Variables in Shell Programming

10.1 Introduction

Variables store data temporarily.

Examples:

- Numbers
- Names
- Strings

10.2 Syntax of Variables

```
variable_name=value
```

Example

```
name="Dhruv"  
echo $name
```

Output

Dhruv

10.3 Rules for Variables

Rule	Explanation
No spaces around =	Required
Variable names case-sensitive	name ≠ NAME
Use \$ to access value	\$name

10.4 Types of Variables

Type	Example
User-defined Variable	x=10
System Variable	\$HOME, \$PATH
Read-only Variable	Fixed value
Local Variable	Valid in current shell

10.5 System Variables

Variable	Meaning
\$HOME	User home directory
\$USER	Current username

\$PWD	Present working directory
\$PATH	Command search path

Example

```
echo $HOME
```

11. Input from User

11.1 read Command

Used to take input from user.

Syntax

```
read variable_name
```

Example

```
#!/bin/bash
```

```
echo "Enter your name:"
```

```
read name
```

```
echo "Hello $name"
```

Sample Output

```
Enter your name:
```

```
Dhruv
```

```
Hello Dhruv
```

12. Simple Shell Scripts

12.1 Script to Display Message

```
#!/bin/bash
```

```
echo "Linux Operating System"
```

12.2 Script for Addition

```
#!/bin/bash
```

```
a=10
```

```
b=20
```

```
sum=$((a+b))
```

```
echo "Sum = $sum"
```

Output

```
Sum = 30
```

12.3 Script for User Input Addition

```
#!/bin/bash
```

```
echo "Enter first number:"
```

```
read a
```

```
echo "Enter second number:"
read b

sum=$((a+b))

echo "Sum = $sum"
```

12.4 Script to Display Current Directory

```
#!/bin/bash
```

```
echo "Current Directory:"
pwd
```

12.5 Script to List Files

```
#!/bin/bash
```

```
echo "Files in Directory:"
ls
```

13. Conditional Statements in Shell

if Statement

Syntax

```
if [ condition ]
then
    commands
fi
```

Example

```
#!/bin/bash
```

```
num=10
```

```
if [ $num -gt 5 ]
then
    echo "Number is greater than 5"
fi
```

Output

```
Number is greater than 5
```

14. Looping in Shell

for Loop Example

```
#!/bin/bash
```

```
for i in 1 2 3
do
    echo $i
done
```

Output

```
1
2
```

15. Real-World Uses of Shell Programming

Area	Usage
System Administration	Backup automation
DevOps	Server scripts
Cybersecurity	Monitoring scripts
Cloud Computing	Deployment automation
File Management	Batch file operations

16. Comparison Table

Shell vs Shell Script

Feature	Shell	Shell Script
Meaning	Command interpreter	File of commands
Execution	One command at time	Multiple commands
Usage	Interactive	Automation
Example	Bash	script.sh

Variables vs Commands

Feature	Variable	Command
Purpose	Store data	Perform operation
Example	x=10	ls
Access	\$x	Direct execution

17. Advantages of Shell Programming

Advantages
Automation support
Easy scripting
Fast task execution
Simple syntax
Powerful administration tool

18. Disadvantages of Shell Programming

Disadvantages
Slower execution
Limited GUI support
Complex debugging
Security risks if poorly written

19. Key Terms

Term	Meaning
Shell	Command interpreter
Script	File containing commands
Variable	Stores data
Bash	Common Linux shell
Interpreter	Executes commands line-by-line
chmod	Change permissions
echo	Display output

20. Important Exam Points

Most Important Questions

1. Explain shell concept in Unix/Linux.
2. What is shell scripting?
3. Explain variables in shell programming.
4. Write a shell script for addition of two numbers.
5. Explain steps to execute shell script.
6. Explain read command with example.
7. Differentiate shell and shell script.

High-Probability Topics

Priority	Topic
Very High	Shell scripting basics
Very High	Variables
High	Script execution steps
High	User input using read
Medium	if statement
Medium	for loop

21. Short Conclusion

Shell programming provides an efficient way to automate tasks in Unix/Linux systems. Using shell scripts, users can execute multiple commands, manage files, process user input, and perform system administration tasks easily. Features such as variables, loops, and conditions make shell scripting a powerful tool in Linux environments.

Directory Structure in Unix/Linux

According to the GTU Operating Systems syllabus, "Directory Structure" is an important topic in Unit-8: Unix/Linux Operating System. Important directories include /, /bin, /etc, /usr, /var, and /home.

1. Introduction

Unix/Linux uses a hierarchical file system structure to organize:

- Files
- Directories
- Programs
- Devices

Everything in Linux is organized under:

- Root directory /

The directory structure helps:

- Efficient file organization
- Security management
- Easy navigation
- Resource management

2. Definition

Term	Definition
Directory	A container used to store files and subdirectories
Directory Structure	Hierarchical organization of directories in Linux

Root Directory

Top-most directory represented by /

3. Directory Hierarchy in Linux

Linux follows:

- Tree-like hierarchical structure

All directories originate from:

- Root directory /

4. Linux Directory Structure Diagram

/ (Root)

├─ bin

├─ etc

├─ home

├─ usr

├─ var

├─ dev

├─ tmp

├─ boot

├─ lib

└─ opt

5. Working Principle of Directory Structure

Basic Working

Root Directory (/)

↓

Subdirectories

↓

Files & Programs

The kernel uses directory hierarchy for:

- File storage
- File retrieval
- Permission management

6. Root Directory (/)

6.1 Definition

The root directory: /

is the top-most directory in Linux.

All files and directories exist under root.

6.2 Importance of Root Directory

Importance

Starting point of file system

Parent of all directories

Essential for system booting

6.3 Example Path

/home/student/file.txt

Here:

- / is root directory

6.4 Advantages

Advantages
Centralized structure
Organized file management

6.5 Disadvantages

Disadvantages
Root corruption affects entire system

7. /bin Directory

7.1 Definition

/bin stands for:

- Binary files

Contains essential Linux commands required for system operation.

7.2 Important Commands Stored

Command	Purpose
ls	List files
cp	Copy files
mv	Move files
rm	Remove files
cat	Display content

7.3 Example

/bin/ls

7.4 Features

Feature	Explanation
Essential commands	Required during boot
Accessible to all users	Public utility commands

7.5 Advantages

Advantages
Quick access to commands
Supports system recovery

8. /etc Directory

8.1 Definition

/etc contains:

- System configuration files

Used for:

- System settings
- User settings
- Service configurations

8.2 Important Files

File	Purpose
/etc/passwd	User information
/etc/hosts	Hostname mapping
/etc/fstab	File system configuration

8.3 Example

cat /etc/passwd

8.4 Features

Feature	Explanation
Stores configuration	System customization
Text-based files	Easy editing

8.5 Advantages

Advantages
Centralized configuration
Easy administration

8.6 Disadvantages

Disadvantages
Wrong editing may crash system

9. /home Directory

9.1 Definition

/home stores:

- Personal files of users

Each user gets separate directory.

9.2 Example Structure

/home

```

├── student
├── admin
└── user1

```

9.3 Example Path

/home/student/Documents

9.4 Features

Feature	Explanation
User-specific storage	Personal data
Separate directories	Better security

9.5 Advantages

Advantages
User data isolation
Better organization

9.6 Disadvantages

Disadvantages

Large user data consumes storage

10. /usr Directory

10.1 Definition

/usr contains:

- User programs
- Libraries
- Documentation
- Applications

It is one of the largest directories.

10.2 Structure of /usr

/usr

```
├── bin
├── lib
├── local
└── share
```

10.3 Important Subdirectories

Subdirectory	Purpose
/usr/bin	User commands
/usr/lib	Libraries
/usr/share	Shared files
/usr/local	Locally installed software

10.4 Features

Feature	Explanation
Stores applications	Installed software
Shared resources	Libraries and manuals

10.5 Advantages

Advantages
Organized software storage
Easy software management

10.6 Disadvantages

Disadvantages
Large disk usage

11. /var Directory

11.1 Definition

/var stands for:

- Variable data

Contains frequently changing files.

11.2 Stored Data

Type	Example
Logs	System logs

Cache	Temporary cache
Mail	Email data
Spool	Printer queue

11.3 Example Structure

/var

```

├── log
├── cache
├── mail
└── spool

```

11.4 Important Example

/var/log

contains system log files.

11.5 Features

Feature	Explanation
Dynamic files	Frequently updated
Logging support	System monitoring

11.6 Advantages

Advantages
Useful for debugging
Supports system monitoring

11.7 Disadvantages

Disadvantages
Logs may consume large storage

12. Other Important Directories

Directory	Purpose
/dev	Device files
/tmp	Temporary files
/boot	Boot loader files
/lib	Shared libraries
/opt	Optional software

13. Complete Directory Hierarchy Example

```

/
├── bin
├── etc
├── home
├── └── student
├── usr
├── └── bin
├── └── lib
└── var

```

| └─ log
└─ tmp

14. File Path Concept

Absolute Path

Starts from root /

Example: /home/student/file.txt

Relative Path

Starts from current directory

Example: Documents/file.txt

15. Comparison Table

Important Linux Directories

Directory	Purpose	Example Content
/	Root directory	Entire file system
/bin	Essential commands	ls, cp
/etc	Configuration files	passwd
/home	User files	Documents
/usr	Programs/libraries	Applications
/var	Variable data	Logs

/bin vs /usr/bin

Feature	/bin	/usr/bin
Purpose	Essential commands	Additional user commands
Required for Boot	Yes	No
Example	ls	gcc

16. Advantages of Linux Directory Structure

Advantages
Organized storage
Better security
Efficient navigation
Easy backup
Standardized hierarchy

17. Disadvantages

Disadvantages
Difficult for beginners
Permission complexity
Large hierarchy structure

18. Real-World Examples

Area	Usage
Web Servers	Store website data
Linux Desktop	User file organization
Cloud Servers	Log management
System Administration	Configuration management

Cybersecurity	Log analysis
---------------	--------------

19. Key Terms

Term	Meaning
Root Directory	Top-level directory /
Subdirectory	Directory inside another
Path	Location of file/directory
Absolute Path	Full path from root
Relative Path	Path from current directory
Configuration File	System settings file

20. Important Exam Points

Most Important Questions

1. Explain Linux directory structure.
2. Explain root directory with diagram.
3. Explain /bin, /etc, /usr, /var, and /home.
4. Differentiate absolute and relative path.
5. Explain directory hierarchy in Linux.
6. Explain importance of /etc.
7. Explain user directories in Linux.

High-Probability Topics

Priority	Topic
Very High	Root directory /
Very High	/etc
High	/home
High	/usr
High	/var
Medium	/bin

21. Short Conclusion

The Unix/Linux directory structure follows a hierarchical tree-based organization starting from the root directory /. Important directories such as /bin, /etc, /home, /usr, and /var help organize commands, configuration files, user data, applications, and system logs efficiently. This structured design provides better security, management, and reliability in Unix/Linux systems.

System Administration in Unix/Linux

According to the GTU Operating Systems syllabus, "System Administration" is an important topic in Unit-8: Unix/Linux Operating System. It includes User Management, File Permissions, and Basic System Monitoring.

1. Introduction

System administration is the process of:

- Managing users
- Controlling permissions
- Monitoring system resources
- Maintaining system security and performance

A system administrator ensures:

- Smooth operation of Linux system
- Security of files and users
- Proper resource utilization

2. Definition

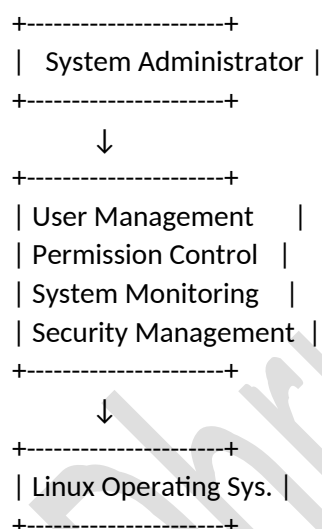
Term	Definition
System Administration	Management and maintenance of operating system resources and users
Administrator (root user)	User having complete system control
System Monitoring	Observing system performance and resources

3. Role of System Administrator

A Linux administrator performs:

- User account management
- Permission control
- Resource monitoring
- Software installation
- Backup management
- Security maintenance

4. Basic System Administration Architecture



5. User Management

5.1 Introduction

Linux is a:

- Multiuser operating system

Many users can work on the same system simultaneously.

Each user has:

- Username
- Password
- Home directory
- Permissions

5.2 Types of Users

User Type	Description
Root User	Full system access
Normal User	Limited permissions
System User	Used by services/processes

5.3 Root User

Root user:

- Has complete administrative privileges
- Can modify any file/system setting

Important Note

Root user is also called Superuser.

5.4 User Management Commands

Command	Purpose
useradd	Create user
passwd	Set password
userdel	Delete user
usermod	Modify user
who	Show logged-in users

5.5 Creating User

Syntax

```
sudo useradd username
```

Example

```
sudo useradd student
```

5.6 Setting Password

Syntax

```
sudo passwd username
```

Example

```
sudo passwd student
```

5.7 Deleting User

Syntax

```
sudo userdel username
```

Example

```
sudo userdel student
```

5.8 Viewing Logged-in Users

```
who
```

Sample Output

```
Student pts/0
admin pts/1
```

5.9 Advantages of User Management

Advantages
Better security
Multiuser support
Separate user environments
Controlled access

5.10 Disadvantages

Disadvantages
Complex administration
Permission conflicts

6. File Permissions

6.1 Introduction

Linux protects files using:

- Permissions

Permissions decide:

- Who can read
- Who can write
- Who can execute files

6.2 Permission Categories

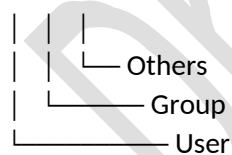
Category	Meaning
User (u)	File owner
Group (g)	Group members
Others (o)	Remaining users

6.3 Permission Types

Symbol	Meaning
r	Read
w	Write
x	Execute

6.4 Permission Structure

rwx



6.5 Example of Permissions

```
-rwxr-xr--
```

Meaning

Section	Permission
User	rwx
Group	r-X
Others	r--

6.6 chmod Command

Used to:

- Change permissions

Syntax

chmod permissions filename

Example

chmod 755 file.sh

6.7 Numeric Permission Method

Number	Permission
7	rwX
6	rw-
5	r-X
4	r--

Example

755 = rwx r-x r-x

6.8 chown Command

Used to:

- Change file owner

Syntax

chown owner filename

Example

chown student file.txt

6.9 Advantages of File Permissions

Advantages
Security protection
Prevents unauthorized access
Multiuser safety
Controlled execution

6.10 Disadvantages

Disadvantages
Permission complexity
Incorrect permissions cause issues

7. Basic System Monitoring

7.1 Introduction

System monitoring means observing:

- CPU usage
- Memory usage
- Running processes
- Disk usage
- System performance

7.2 Purpose of Monitoring

Purpose
Detect performance problems
Monitor resources
Improve stability
Detect abnormal activity

7.3 Important Monitoring Commands

Command	Purpose
top	Real-time process monitoring
ps	Display running processes
free	Memory usage
df	Disk space usage
du	Directory size
uptime	System running time

7.4 top Command

Displays:

- CPU usage
- RAM usage
- Active processes

Syntax

top

Features

Feature	Explanation
Dynamic display	Real-time updates
Process monitoring	Shows active processes
CPU monitoring	Resource tracking

7.5 ps Command

Displays:

- Running processes

Syntax

ps

Example

ps -aux

7.6 free Command

Displays:

- RAM usage

Syntax

free -h

Sample Output

Total RAM
Used RAM
Free RAM

7.7 df Command

Displays:

- Disk space usage

Syntax

df -h

7.8 du Command

Displays:

- Directory/file size

Syntax

du -sh folder

7.9 uptime Command

Displays:

- System running time
- Load average

Syntax

uptime

8. Real-World Administration Tasks

Task	Tool/Command
Add user	useradd
Change password	passwd
Monitor CPU	top
Check RAM	free
Check storage	df
Change permissions	chmod

9. Working Principle of System Administration

Administrator



Linux Commands



Kernel



System Resources Managed

10. Comparison Tables

Root User vs Normal User

Feature	Root User	Normal User
Permissions	Full access	Limited access

System Changes	Allowed	Restricted
Security Risk	High	Lower
Usage	Administration	Daily tasks

chmod vs chown

Command	Purpose
chmod	Change permissions
chown	Change ownership

ps vs top

Feature	ps	top
Output	Static	Dynamic
Monitoring	Single snapshot	Real-time
Usage	Process listing	Live monitoring

11. Advantages of System Administration

Advantages
Better system security
Resource management
Stable operation
Multiuser support
Performance monitoring

12. Disadvantages

Disadvantages
Requires technical knowledge
Human configuration errors
Security risk with root misuse

13. Key Terms

Term	Meaning
Root User	Superuser with full access
Permission	Access control mechanism
Process	Program in execution
Monitoring	Observing system performance
Ownership	User controlling file
chmod	Permission change command
chown	Ownership change command

14. Important Exam Points

Most Important Questions

1. Explain user management in Linux.
2. Explain Linux file permissions.
3. Explain chmod command with example.
4. Explain system monitoring commands.
5. Differentiate root user and normal user.
6. Explain top and ps.
7. Explain permission categories in Linux.

15. Short Conclusion

System administration in Unix/Linux involves managing users, controlling file permissions, and monitoring system resources. Tools such as `useradd`, `chmod`, `top`, and `ps` help administrators maintain security, performance, and stability of Linux systems. Proper administration ensures efficient and secure operation of multiuser environments.

DhruvPatel16120